

LIS2DH12 PR — priorities from a real deployment

Feedback for the in-flight LIS2DH12 PR (esphome#14788) based on production use of the component on an ESP32-C6 device alongside a stepper driver, I²C temperature sensor, GPIO buttons, and several interval blocks.

Top priorities, ranked

1. Stop blocking the main loop in `update()`

The current synchronous DRDY-polling loop produces this on every read:

```
[W][component:522]: lis2dh12_base took a long time for an operation (174 ms), max is 36
```



This is the single biggest issue and is covered in detail below.

2. Expose hardware interrupts in YAML

The chip's killer features for ESPHome users are the INT1/INT2 sources: activity/inactivity (wake-up threshold), 6D/4D orientation, single/double-tap, free-fall. Mapping these to `binary_sensor` children would obsolete a lot of polling-based application code. Even just **activity + 6D orientation** would cover the majority of use cases (motion-triggered tasks, tamper detection, tilt detection).

3. Expose the embedded high-pass filter (CTRL_REG2)

Lets the chip subtract DC gravity in hardware so the output IS the AC / vibration component. Removes the need for `|sqrt(x2 + y2 + z2) - 1g|` gymnastics in user lambdas, and makes the downstream interrupt thresholds (#2 above) work on filtered data instead of raw.

4. FIFO mode (32-sample burst)

One I²C transaction per ~30 samples instead of one per sample. Eliminates the per-sample DRDY-poll overhead entirely and lets people sample at high ODR without saturating the bus.

5. Quieter default logging

Currently emits ~50 `i2c.idf` lines per accelerometer read at DEBUG — drowns the bus log for anyone debugging other components. Move the per-register transaction chatter to VERBOSE and keep DEBUG limited to state changes (ODR set, range set, interrupt fired, etc.).

6. Stable YAML schema before merge

I'm pinned to a commit SHA in production because the schema can shift between pushes. A short "schema is frozen for the merge" signal would let me drop the SHA pin.

If time-constrained:

- Do only one → **#1 (main-loop blocking)**. By far the most painful in real deployments because it perturbs every other component on the device.
- Do two → add **#2 (interrupts)**. That's the feature gap that keeps the polling architecture relevant in the first place.

The rest (auto-address detect, self-test mode, sleep mode) are nice-to-haves and not worth listing if you're time-constrained.

Deep dive on #1 — main-loop blocking

What's actually happening

ESPHome runs as a cooperative single-threaded main loop on ESP32 — every component's `loop()` and `update()` execute back-to-back on the same task. A "loop tick" is normally ~16 ms. When one component blocks, nothing else gets to run: button polling, sensor publishes, interval blocks, cover state transitions — all stalled.

The current `lis2dh12_base::update()` looks (functionally) like this:

```
void update() override {
    // Spin on the data-ready bit until the chip says new data is ready
    while (!(read_register(STATUS_REG) & ZYXDA_MASK)) {
        // busy wait - every iteration is one I²C transaction (~1-3 ms)
    }
    uint8_t buf[6];
    read_burst(OUT_X_L | AUTO_INC, buf, 6);
    publish_x_y_z(buf);
}
```

That `while` loop is the killer. You can see it in the raw bus log — dozens of `0x18 TX 39 RX ..` (status reads) before the final `0x18 TX A8 RX ..` (the 6-byte X/Y/Z burst).

Why 174 ms is bad

- The main-loop watchdog warning (`max is 30 ms`) is the visible symptom, but the real cost is **other components getting starved**. 174 ms = ~10 missed main-loop iterations.
- Anything timing-sensitive on the main loop suffers: GPIO debounce filters can miss edges, interval blocks fire late, motion-completion polls run with jitter close to their own period.
- The stepper isn't affected on my board — `tmc2209` runs the actual step generation on a dedicated FreeRTOS task. So the motor keeps moving smoothly, but the supervisor logic around it lags.
- The blocking time is **variable**. It depends on where in the chip's sample cycle the `update()` call happened to land, plus any I²C contention from other devices on the bus. So 174 ms one time, 50 ms next, 300 ms occasionally — unpredictable jitter is worse for the overall system than a steady delay would be.

The three fixes, ranked

1. DRDY-interrupt-driven (the right answer)

The chip has `CTRL_REG3.I1_ZYXDA` ; set it and INT1 asserts the instant new XYZ data is available. Component config takes an `interrupt_pin` ; an ISR sets a `volatile bool data_ready` flag; `loop()` (not `update()`) checks the flag and only then issues the burst read. Net cost per sample: ~one 6-byte I²C transaction (~1.5 ms), latency from chip-ready to publish is sub-millisecond. Zero polling, zero blocking.

```
void setup() override {
    write_register(CTRL_REG3, I1_ZYXDA);           // route DRDY → INT1
    attachInterrupt(int_pin_, isr_, RISING);
}
void loop() override {
    if (!data_ready_) return;
    data_ready_ = false;
    uint8_t buf[6];
    read_burst(OUT_X_L | AUTO_INC, buf, 6);
    publish_x_y_z(buf);
}
```

2. Skip-and-retry (acceptable fallback for users who don't wire INT1)

`update()` reads STATUS *once*. If `ZYXDA == 0`, return immediately and try again next tick. The penalty is that occasional ticks publish nothing — completely fine because the application either polls cached values at its own cadence anyway, or wants new-data-or-nothing semantics (also fine).

```
void update() override {
    if (!(read_register(STATUS_REG) & ZYXDA_MASK)) return;
    uint8_t buf[6];
    read_burst(OUT_X_L | AUTO_INC, buf, 6);
    publish_x_y_z(buf);
}
```

This alone eliminates the warning. Single STATUS read = ~1.5 ms, comfortably under 30 ms.

3. Yield between polls (worst of the three, but better than nothing)

Call `App::feed_wdt()` and a brief `delay(0) / taskYIELD()` between status polls. Doesn't actually unblock the main loop — other components still wait — but the hardware watchdog and the IDF task scheduler get to run. This is what you do when you can't change the polling architecture. Avoid if the other two are feasible.

Recommended path

#2 immediately — one-line change, fixes the warning for everyone.

#1 as the proper long-term solution — optional `interrupt_pin:` config, gated on whether the user wired INT1. Both can coexist: interrupt mode when configured, skip-and-retry when not.

ESPHome's `LIS3DH` integration (different chip, related family) uses the skip-and-retry pattern and doesn't have this problem — there's an in-tree precedent to reference.

Why this matters beyond cosmetics

Anyone building real applications on the chip — door/window sensors, vibration monitors, anti-tamper systems — is going to mix it with other components. A 174 ms hiccup every 200 ms means the chip is effectively unusable alongside fast button debouncing, hardware-timer-driven outputs, or any other timing-sensitive logic. Right now the only safe deployments are ones where LIS2DH12 is the *only* time-sensitive thing on the device.